Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Кафедра инженерной психологии и эргономики

Специальность «Программная инженерия»

Учебная дисциплина «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе №5
«Файловое хранилище»

Подготовил Сулимов А.А.

Проверил: Болтак С.В.

Минск 2026

Задание: Необходимо реализовать удаленное файловое хранилище, предоставляющее REST API по протоколу HTTP. При реализации разрешается использовать веб-фреймворк, реализующий HTTP-роутинг и работу с HTTP-вызовами.

Путь URL запроса определяет логическое расположение файла в хранилище:

http://storage.com/path/to/file.txt - описывает файл file.txt, находящийся по пути /path/to.

Список поддерживаемых функций:

- загрузка файла в хранилище с перезаписью с помощью метода PUT;
- получение файла из хранилища с помощью метода GET;
- получение списка файлов каталога с помощью метода GET (в качестве формата данных для ответа рекомендуется использовать JSON);
- получение информации о файле (размер в байтах и дата последнего изменения) в виде HTTP-заголовков без получения содержимого файла с помощью метода HEAD;
- удаление файла/каталога из хранилища с помощью метода DELETE.

Необходимо корректно использовать коды состояния HTTP.

Для тестирования созданной службы использовать утилиту curl или аналоги (Postman, HTTPie, etc). Получение файла или содержимого каталога также должно быть возможным через браузер.

Исходные данные:

На рисунках 1-2 представлена работа метода PUT для файла, находящегося в последовательности папок и работа Wireshark.

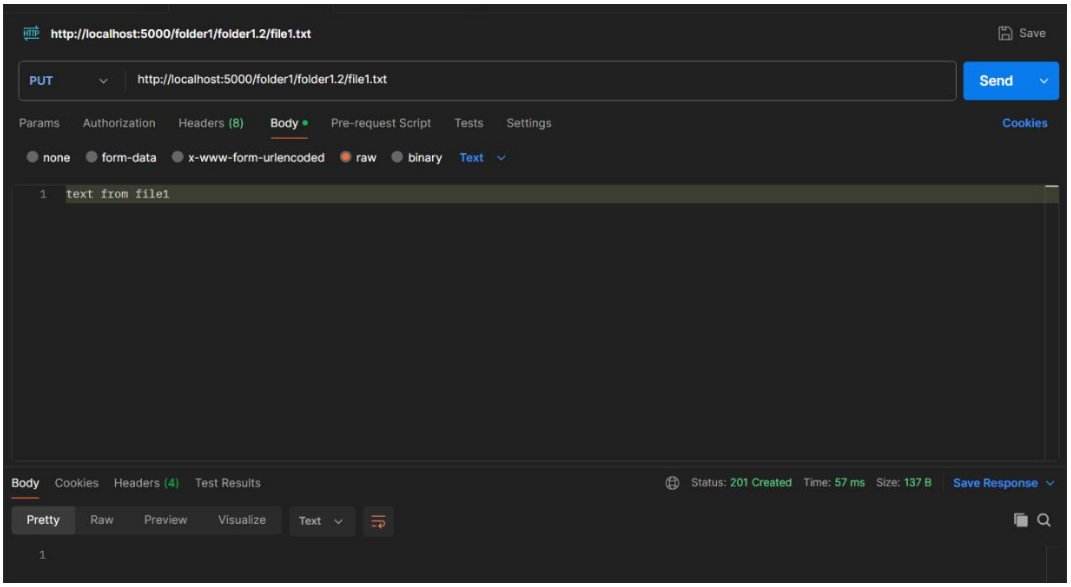
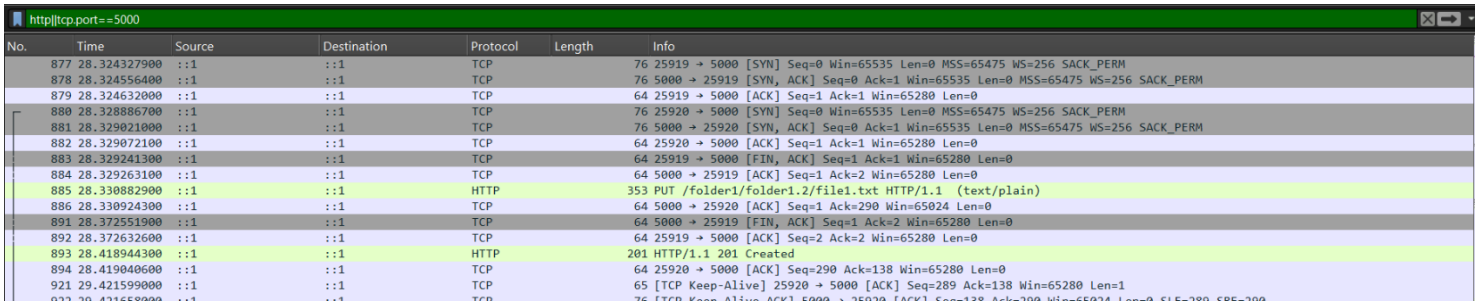


Рисунок 1



http||tcp.port==5000

No.	Time	Source	Destination	Protocol	Length	Info
877	28.324327900	::1	::1	TCP	76	25919 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
878	28.324556400	::1	::1	TCP	76	5000 → 25919 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
879	28.324632000	::1	::1	TCP	64	25919 → 5000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
880	28.328886700	::1	::1	TCP	76	25920 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
881	28.329021000	::1	::1	TCP	76	5000 → 25920 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
882	28.329072100	::1	::1	TCP	64	25920 → 5000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
883	28.329241300	::1	::1	TCP	64	25919 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
884	28.329263100	::1	::1	TCP	64	5000 → 25919 [ACK] Seq=1 Ack=2 Win=65280 Len=0
885	28.330882900	::1	::1	HTTP	353	PUT /folder1/folder1.2/file1.txt HTTP/1.1 (text/plain)
886	28.330924300	::1	::1	TCP	64	5000 → 25920 [ACK] Seq=1 Ack=290 Win=65024 Len=0
891	28.372551900	::1	::1	TCP	64	5000 → 25919 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
892	28.372632600	::1	::1	TCP	64	25919 → 5000 [ACK] Seq=2 Ack=2 Win=65280 Len=0
893	28.418944300	::1	::1	HTTP	201	HTTP/1.1 201 Created
894	28.419040600	::1	::1	TCP	64	25920 → 5000 [ACK] Seq=290 Ack=138 Win=65280 Len=0
921	29.421599000	::1	::1	TCP	65	[TCP Keep-Alive] 25920 → 5000 [ACK] Seq=289 Ack=138 Win=65280 Len=1

Рисунок 2 – трехкратное рукопожатие и метод PUT.

На рисунках 3-4 представлена работа метода PUT для файла в одной папке и работа Wireshark.

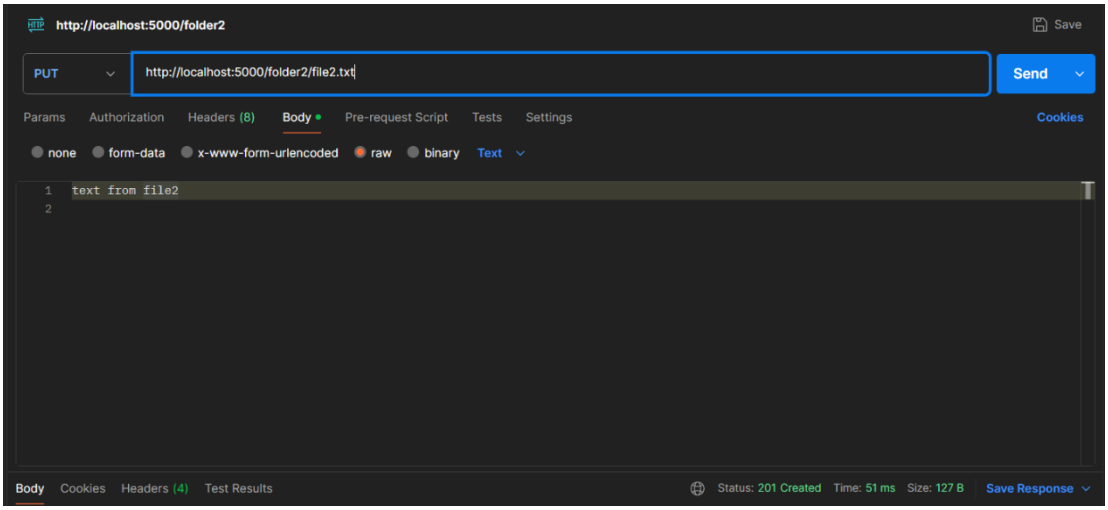
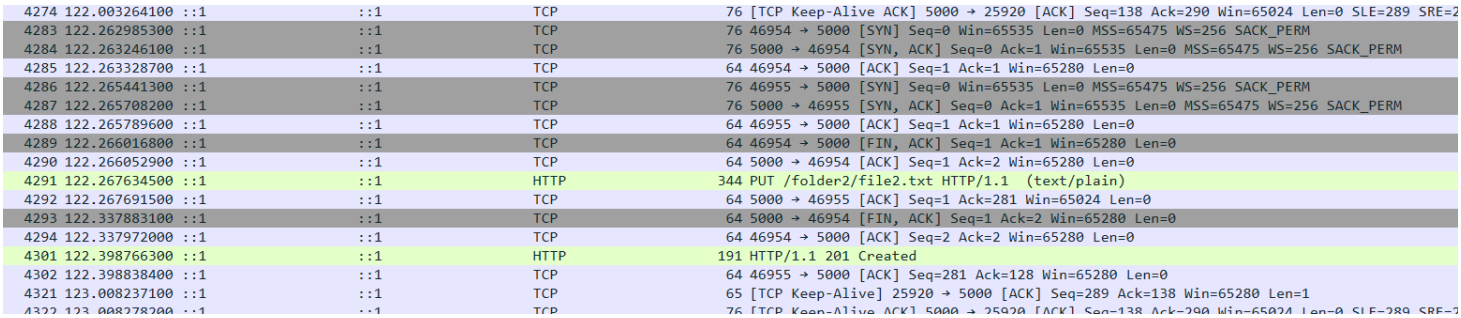


Рисунок 3



4274	122.003264100	::1	::1	TCP	76	[TCP Keep-Alive ACK] 5000 → 25920 [ACK] Seq=138 Ack=290 Win=65024 Len=0 SLE=289 SRE=2
4283	122.262985300	::1	::1	TCP	76	46954 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
4284	122.263246100	::1	::1	TCP	76	5000 → 46954 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
4285	122.263328700	::1	::1	TCP	64	46954 → 5000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
4286	122.265441300	::1	::1	TCP	76	46955 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
4287	122.265708200	::1	::1	TCP	76	5000 → 46955 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
4288	122.265789600	::1	::1	TCP	64	46955 → 5000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
4289	122.266016800	::1	::1	TCP	64	46954 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
4290	122.266052900	::1	::1	TCP	64	5000 → 46954 [ACK] Seq=1 Ack=2 Win=65280 Len=0
4291	122.267634500	::1	::1	HTTP	344	PUT /folder2/file2.txt HTTP/1.1 (text/plain)
4292	122.267691500	::1	::1	TCP	64	5000 → 46955 [ACK] Seq=1 Ack=281 Win=65024 Len=0
4293	122.337883100	::1	::1	TCP	64	5000 → 46954 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
4294	122.337972000	::1	::1	TCP	64	46954 → 5000 [ACK] Seq=2 Ack=2 Win=65280 Len=0
4301	122.398766300	::1	::1	HTTP	191	HTTP/1.1 201 Created
4302	122.398838400	::1	::1	TCP	64	46955 → 5000 [ACK] Seq=281 Ack=128 Win=65280 Len=0
4321	123.008237100	::1	::1	TCP	65	[TCP Keep-Alive] 25920 → 5000 [ACK] Seq=289 Ack=138 Win=65280 Len=1

Рисунок 4 – работа Wireshark для метода PUT

На рисунках 5-6 представлена работа метода PUT для файла в уже существующую папку и работа Wireshark.

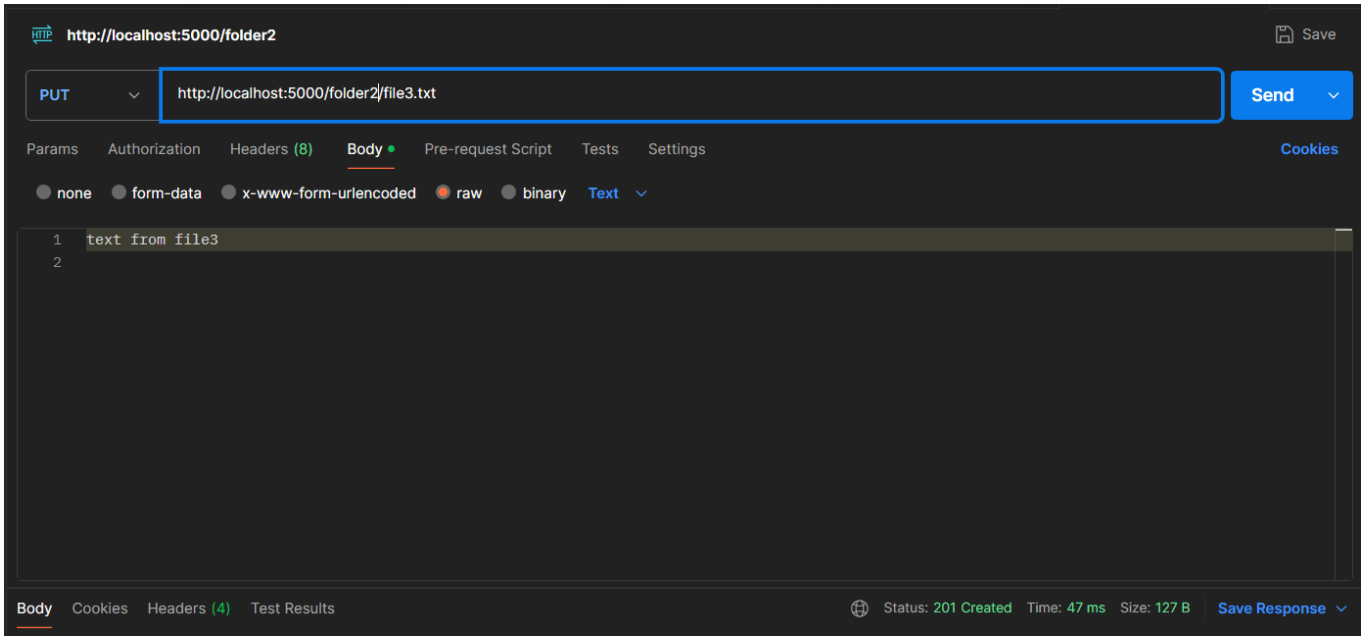
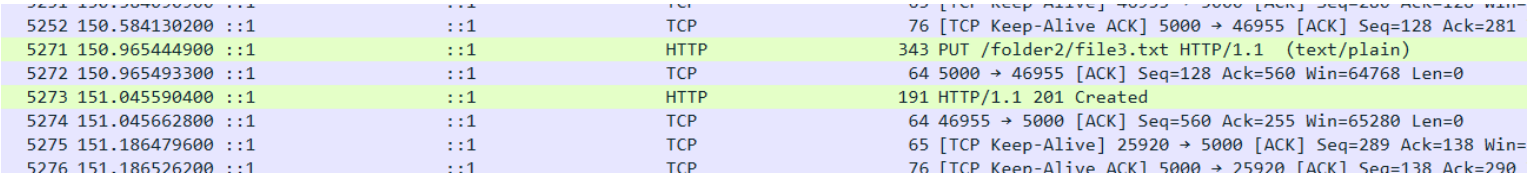


Рисунок 5



5252	150.584130200	::1	::1	TCP	76 [TCP Keep-Alive ACK] 5000 → 46955 [ACK] Seq=128 Ack=281
5271	150.965444900	::1	::1	HTTP	343 PUT /folder2/file3.txt HTTP/1.1 (text/plain)
5272	150.965493300	::1	::1	TCP	64 5000 → 46955 [ACK] Seq=128 Ack=560 Win=64768 Len=0
5273	151.045590400	::1	::1	HTTP	191 HTTP/1.1 201 Created
5274	151.045662800	::1	::1	TCP	64 46955 → 5000 [ACK] Seq=560 Ack=255 Win=65280 Len=0
5275	151.186479600	::1	::1	TCP	65 [TCP Keep-Alive] 25920 → 5000 [ACK] Seq=289 Ack=138 Win=
5276	151.186526200	::1	::1	TCP	76 [TCP Keep-Alive ACK] 5000 → 25920 [ACK] Seq=138 Ack=290

Рисунок 6 – работа Wireshark для метода PUT

На рисунках 7-8 представлена работа метода GET для хранилища и работа Wireshark.

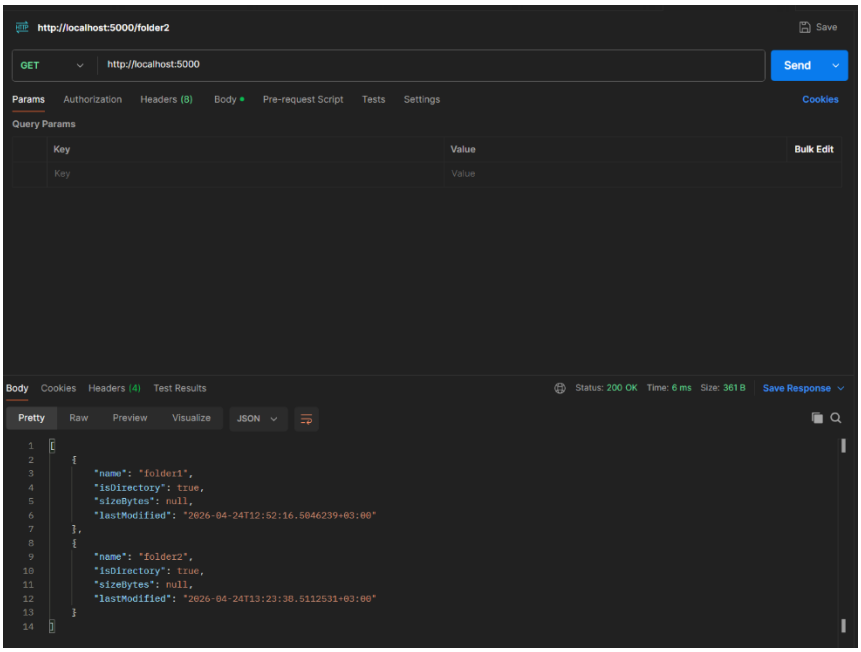


Рисунок 7

5998	171.309582400	::1	::1	TCP	76 [TCP Keep-Alive ACK] 5000 → 25920 [ACK] Seq=138 Ack=290 Win=6502
6003	171.717449600	::1	::1	HTTP	326 GET / HTTP/1.1 (text/plain)
6004	171.717538900	::1	::1	TCP	64 5000 → 46955 [ACK] Seq=255 Ack=822 Win=64512 Len=0
6007	171.761557800	::1	::1	TCP	431 5000 → 46955 [PSH, ACK] Seq=255 Ack=822 Win=64512 Len=367 [TCP P
6008	171.761645800	::1	::1	TCP	64 46955 → 5000 [ACK] Seq=822 Ack=622 Win=64768 Len=0
6009	171.761746300	::1	::1	HTTP/JSON	69 HTTP/1.1 200 OK , JSON (application/json)
6010	171.761771800	::1	::1	TCP	64 46955 → 5000 [ACK] Seq=822 Ack=627 Win=64768 Len=0
6019	172.310235300	::1	::1	TCP	65 [TCP Keep-Alive] 25920 → 5000 [ACK] Seq=289 Ack=138 Win=65280 Le

Рисунок 8 – работа метода GET

На рисунках 9-10 представлена работа метода GET для текстового файла и работа Wireshark.

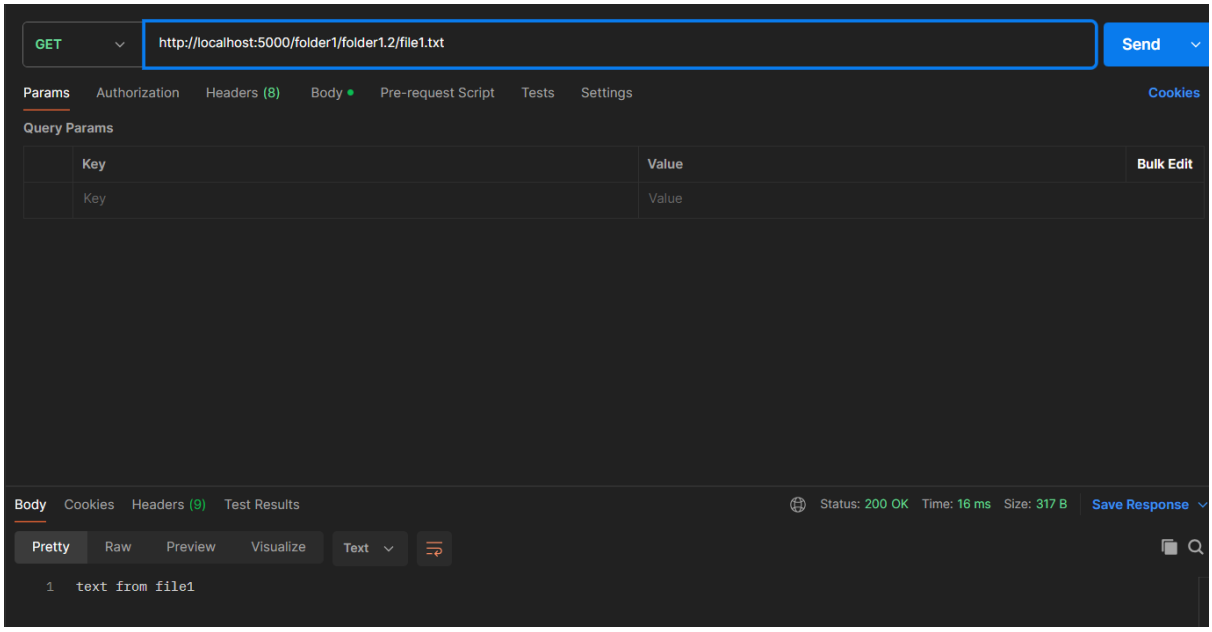


Рисунок 9

7045	201.512908900	::1	::1	TCP	76 [TCP Keep-Alive ACK] 5000 → 25920 [ACK] Seq=138 Ack=290 Win=65024 Len=0
7062	201.880028800	::1	::1	HTTP	353 GET /folder1/folder1.2/file1.txt HTTP/1.1 (text/plain)
7063	201.880083600	::1	::1	TCP	64 5000 → 46955 [ACK] Seq=894 Ack=1390 Win=64000 Len=0
7064	201.930220600	::1	::1	HTTP	381 HTTP/1.1 200 OK (text/plain)
7065	201.930287200	::1	::1	TCP	64 46955 → 5000 [ACK] Seq=1390 Ack=1211 Win=64256 Len=0
7074	202.516970700	::1	::1	TCP	65 [TCP Keep-Alive] 25920 → 5000 [ACK] Seq=289 Ack=138 Win=65280 Len=1
7075	202.517016600	::1	::1	TCP	76 [TCP Keep-Alive ACK] 5000 → 25920 [ACK] Seq=138 Ack=290 Win=65024 Len=0

Рисунок 10

На рисунках 11-12 представлена работа метода HEAD для текстового файла.

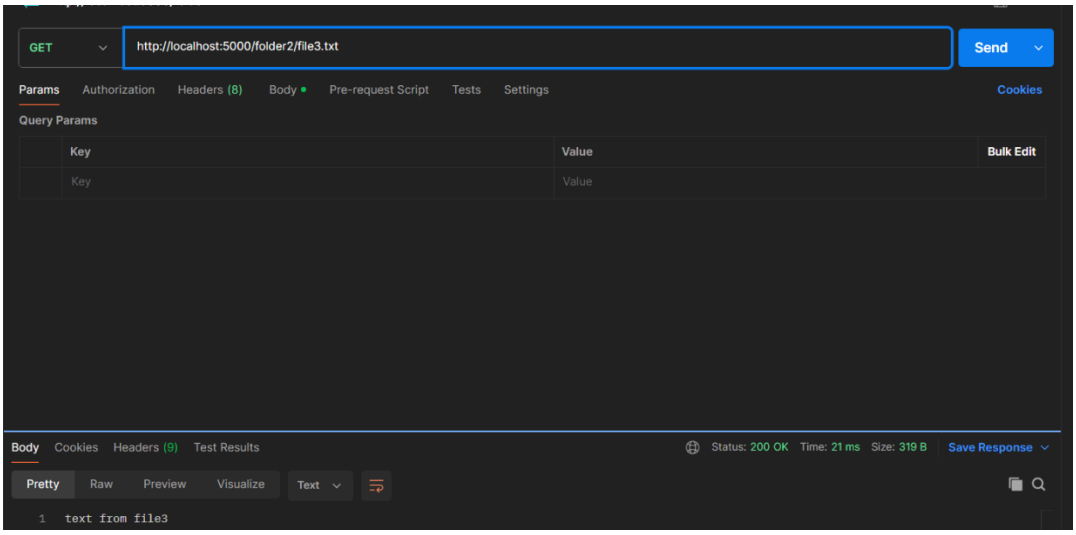


Рисунок 11

8697	249.841100600	::1	::1	TCP	76 [TCP Keep-Alive ACK] 5000 → 25920 [ACK] Seq=138 Ack=290 Win=
8716	250.296829400	::1	::1	HTTP	344 HEAD /folder2/file3.txt HTTP/1.1 (text/plain)
8717	250.296923500	::1	::1	TCP	64 5000 → 46955 [ACK] Seq=1211 Ack=1670 Win=63744 Len=0
8718	250.336590100	::1	::1	HTTP	203 HTTP/1.1 200 OK
8719	250.336663100	::1	::1	TCP	64 46955 → 5000 [ACK] Seq=1670 Ack=1350 Win=64000 Len=0

Рисунок 12 – работа Wireshark для метода HEAD

На рисунках 13-14 представлена работа метода DELETE для текстового файла и работа Wireshark.

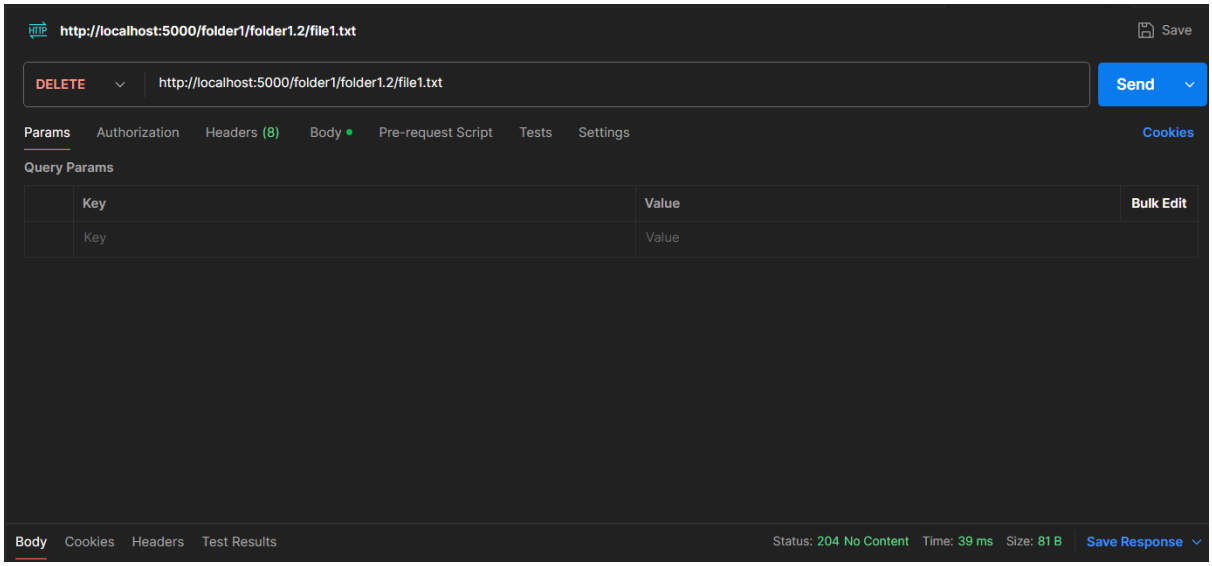


Рисунок 13

9759	280.555854700	::1	::1	TCP	76 [TCP Keep-Alive ACK] 5000 → 46955 [ACK] Seq=1350 Ack=1670 Win=63744 Len=
9774	280.732187500	::1	::1	HTTP	356 DELETE /folder1/folder1.2/file1.txt HTTP/1.1 (text/plain)
9775	280.732241300	::1	::1	TCP	64 5000 → 46955 [ACK] Seq=1350 Ack=1962 Win=63488 Len=0
9776	280.767789300	::1	::1	HTTP	145 HTTP/1.1 204 No Content
9777	280.767877200	::1	::1	TCP	64 46955 → 5000 [ACK] Seq=1962 Ack=1431 Win=64000 Len=0
9790	281.064548700	::1	::1	TCP	65 [TCP Keep-Alive] 25920 → 5000 [ACK] Seq=289 Ack=138 Win=65280 Len=1
9791	281.064591500	::1	::1	TCP	76 [TCP Keep-Alive ACK] 5000 → 25920 [ACK] Seq=138 Ack=290 Win=65024 Len=0

Рисунок 14 – работа Wireshark для метода DELETE

На рисунках 15-16 представлена работа метода DELETE для папки

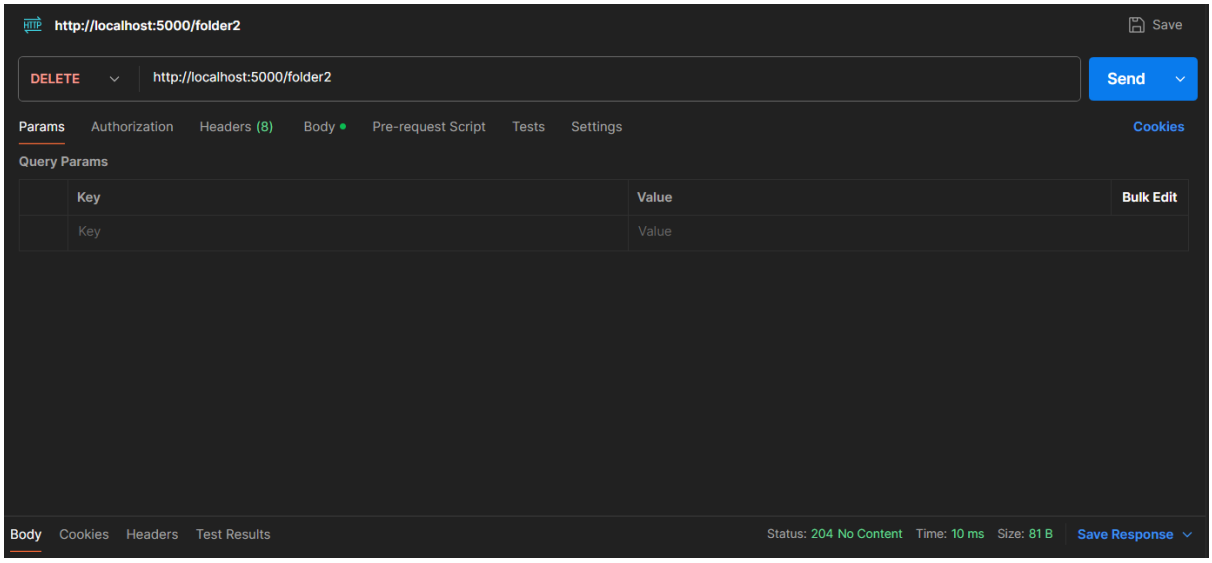


Рисунок 15

10393	298.155625600	::1	::1	TCP	76 [TCP Keep-Alive ACK] 5000 → 25920 [ACK] Seq=138 Ack=290
10394	298.244162600	::1	::1	HTTP	336 DELETE /folder2 HTTP/1.1 (text/plain)
10395	298.244240800	::1	::1	TCP	64 5000 → 46955 [ACK] Seq=1431 Ack=2234 Win=63232 Len=0
10396	298.251002200	::1	::1	HTTP	145 HTTP/1.1 204 No Content
10397	298.251097400	::1	::1	TCP	64 46955 → 5000 [ACK] Seq=2234 Ack=1512 Win=64000 Len=0

Рисунок 16 – работа Wireshark

На рисунках 17-19 представлена работа браузера.

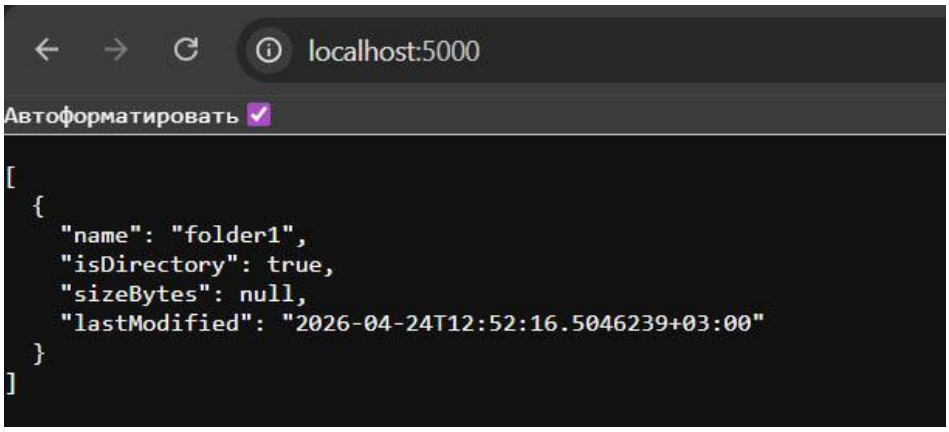


Рисунок 17

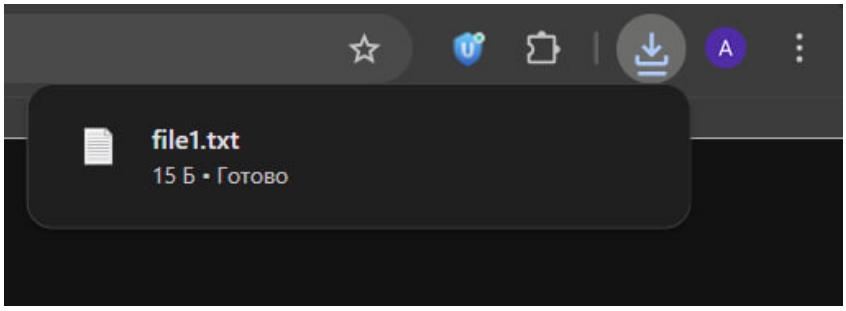


Рисунок 18 – скачивание файла

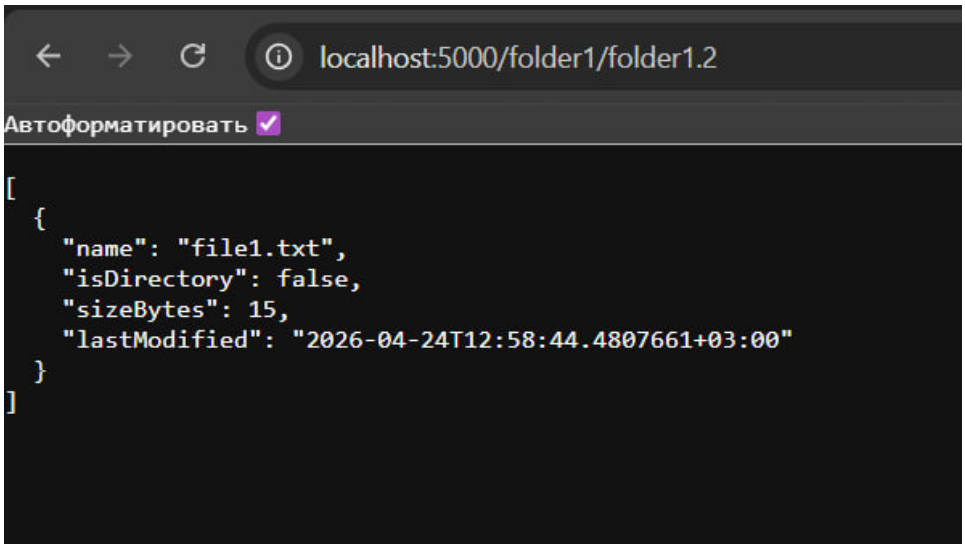


Рисунок 19

Вывод

В ходе выполнения лабораторной работы было успешно спроектировано и реализовано удаленное файловое хранилище, предоставляющее REST API по протоколу HTTP.

Программный продукт реализует полный цикл управления файловыми ресурсами, включая загрузку с автоматической перезаписью (метод PUT), получение содержимого (метод GET) и удаление объектов (метод DELETE). В архитектуру сервиса интегрирована логика динамического маппинга URL-путей на физическую файловую систему, что позволяет прозрачно работать с вложенными структурами каталогов. Особое внимание уделено реализации метода HEAD, возвращающего метаданные файлов (размер и дату изменения) без передачи тела ответа, что оптимизирует нагрузку на сеть согласно принципам REST.

Для обеспечения информационной безопасности был разработан алгоритм нормализации путей и валидации запросов, исключающий возможность атак типа Path Traversal и ограничивающий доступ пользователя рамками корневой папки хранилища. Дополнительно

реализован механизм управления HTTP-заголовками (Content-Disposition: attachment и Cache-Control), гарантирующий принудительное скачивание файлов браузером и исключающий использование устаревшего кэша.

Тестирование разработанного решения проводилось с помощью утилиты Postman, а также через веб-интерфейс браузера. В ходе проверок была подтверждена корректность обработки статус-кодов (200 OK, 201 Created, 204 No Content) и генерации ответов в формате JSON для листинга каталогов.